МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение

высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 14

Тема: Поведенческие паттерны: Системные паттерны и работа с данными

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки
09.03.04 «Программная
инженерия»

Постников Илья Викторович

Проверил:

Доцент ДЦРСИЭ

Альбекова З. М.

Ставрополь 2026

Цель: изучить принцип работы системного паттерна «Repository» и реализовать прикладную задачу «Графический пароль» с его использованием.

Задание: разработать приложение для работы с графическим паролем и реализовать один из паттернов: Repository, Data Mapper, Active Record, Unit of Work.

Выбран паттерн Repository.

Вариант 8 - Графический пароль

1. Описание паттерна «Repository»

Repository - системный паттерн, который инкапсулирует логику доступа к данным и предоставляет клиенту интерфейс, похожий на коллекцию объектов в памяти. Клиент не знает, где и как хранятся данные (JSON-файл, реляционная БД, удалённый сервис).

Преимущества:

- бизнес-логика не зависит от способа хранения;
- замена хранилища не требует правок в сервисах и UI;
- упрощается тестирование (можно подставить in-memory репозиторий).

В приложении:

- PasswordRecord - доменная сущность;
- PasswordRepository - репозиторий, работающий с graphical_password.json;
- PasswordService - бизнес-логика, использующая репозиторий;
- PasswordController - UI, не знающий о файле.

2. Описание разработанного приложения

Приложение позволяет формировать, сохранять, проверять и сбрасывать графический пароль. Дополнительно ведётся счётчик неудачных попыток входа, который хранится в той же записи репозитория.

Основное окно содержит:

- метку состояния пароля и числа неудачных попыток;
- сетку 3×3;

– текущую последовательность;
– кнопки «Войти» / «Сохранить пароль», «Очистить»;
– переключатель «Вход / Создать пароль»;
– кнопку «Сбросить сохранённый пароль»;
– статусную строку.

Требования к паролю: длина 4–9, без повторов, ячейки 1–9.

3. Диаграмма классов

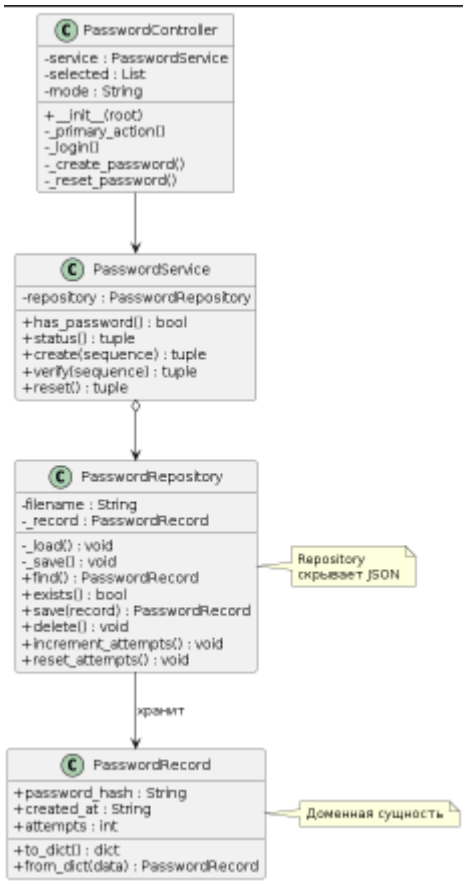


Рисунок 1 - Диаграмма классов приложения «Графический пароль» (Repository)

4. Реализация паттерна «Repository»

Доменная сущность:

```
class PasswordRecord:
    def __init__(self, password_hash, created_at=None, attempts=0):
        self.password_hash = password_hash
        self.created_at = created_at or ""
        self.attempts = attempts
```

```
    def to_dict(self):
        return {
            "password_hash": self.password_hash,
            "created_at": self.created_at,
            "attempts": self.attempts,
        }

    @classmethod
    def from_dict(cls, data):
        return cls(
            password_hash=data.get("password_hash"),
            created_at=data.get("created_at", ""),
            attempts=data.get("attempts", 0),
        )
```

Репозиторий:

```
class PasswordRepository:
    def __init__(self, filename="graphical_password.json"):
        self.filename = filename
        self._record = None
        self._load()

    def _load(self):
        if os.path.exists(self.filename):
            try:
                with open(self.filename, "r", encoding="utf-8") as f:
                    data = json.load(f)
                if "password_hash" in data:
                    self._record = PasswordRecord.from_dict(data)
            except (json.JSONDecodeError, OSError):
                self._record = None
```

```
def _save(self):
    if self._record is None:
        if os.path.exists(self.filename):
            os.remove(self.filename)
        return
    with open(self.filename, "w", encoding="utf-8") as f:
        json.dump(self._record.to_dict(), f, ensure_ascii=False, indent=4)

def find(self):
    return self._record

def exists(self):
    return self._record is not None

def save(self, record):
    self._record = record
    self._save()
    return record

def delete(self):
    self._record = None
    self._save()

def increment_attempts(self):
    if self._record is None:
        return
    self._record.attempts += 1
    self._save()
```

```
    def reset_attempts(self):
        if self._record is None:
            return
        self._record.attempts = 0
        self._save()
```

Сервис использует репозиторий и не знает, что под ним JSON:

```
class PasswordService:
    def __init__(self, repository=None):
        self.repository = repository or PasswordRepository()

    def has_password(self):
        return self.repository.exists()

    def create(self, sequence):
        if self.repository.exists():
            return False, "Пароль уже существует.", None
        record = PasswordRecord(
            password_hash=hash_sequence(sequence),
            created_at=__import__("datetime").datetime.now().strftime("%Y-
%m-%d %H:%M:%S"),
        )
        self.repository.save(record)
        return True, "Пароль сохранён.", record.password_hash

    def verify(self, sequence):
        record = self.repository.find()
        if record is None:
            return False, "Пароль не создан.", None
        if hash_sequence(sequence) == record.password_hash:
            self.repository.reset_attempts()
```

```
            return True, "Пароль верный. Доступ разрешён.", None
        self.repository.increment_attempts()
        return False, "Пароль неверный. Доступ запрещён.", None

    def reset(self):
        self.repository.delete()
    return True, "Пароль удалён.", None
```

Ключевая идея: PasswordService оперирует PasswordRecord и методами репозитория. Детали хранения (имя файла, JSON, удаление файла при пустом репозитории) полностью скрыты.

5. Взаимодействие компонентов

Поток «Создать пароль»:

- PasswordController формирует последовательность и передаёт её в PasswordService.create().
- Сервис проверяет repository.exists().
- Создаёт PasswordRecord и вызывает repository.save(record).
- Репозиторий сериализует запись в graphical_password.json.
- Контроллер получает кортеж (success, message, hash) и обновляет UI.

Поток «Войти»:

- PasswordController передаёт последовательность в PasswordService.verify().
- Сервис вызывает repository.find().
- При совпадении хэшей - repository.reset_attempts(), иначе - repository.increment_attempts().

Контроллер отображает результат и обновляет счётчик попыток.

Таблица 1 - Операции репозитория

Метод репозитория	Назначение
find()	Вернуть текущую запись или None
exists()	Проверить наличие пароля

save(record)	Сохранить запись (создать/обновить)
delete()	Удалить запись и файл
increment_attempts()	Увеличить счётчик неудачных попыток
reset_attempts()	Сбросить счётчик

6. Результаты работы программы

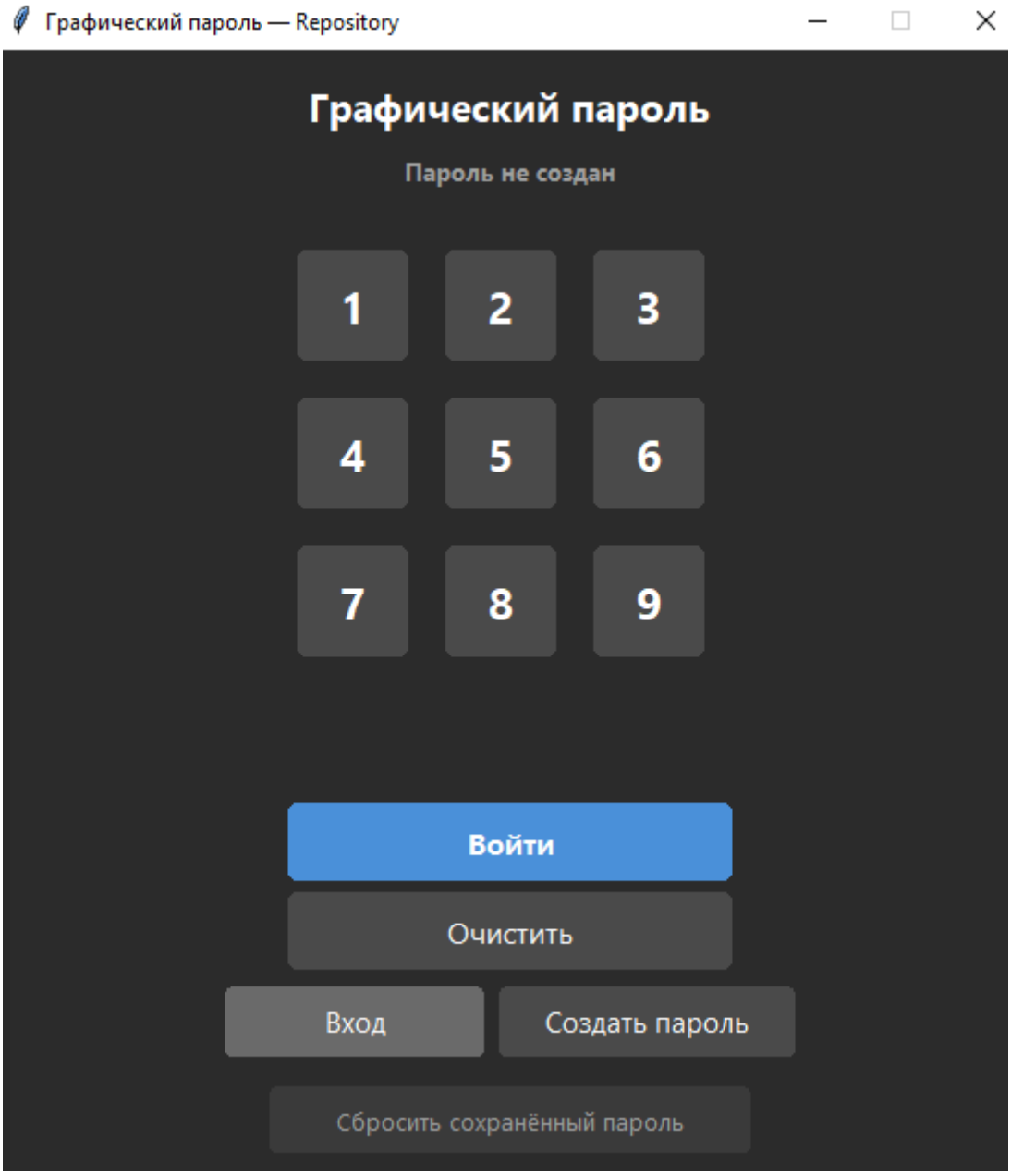


Рисунок 2 - Главное окно.

Метка «Пароль не создан», сетка 3×3, кнопки «Войти», «Очистить», переключатель «Вход / Создать пароль», «Сбросить сохранённый пароль».

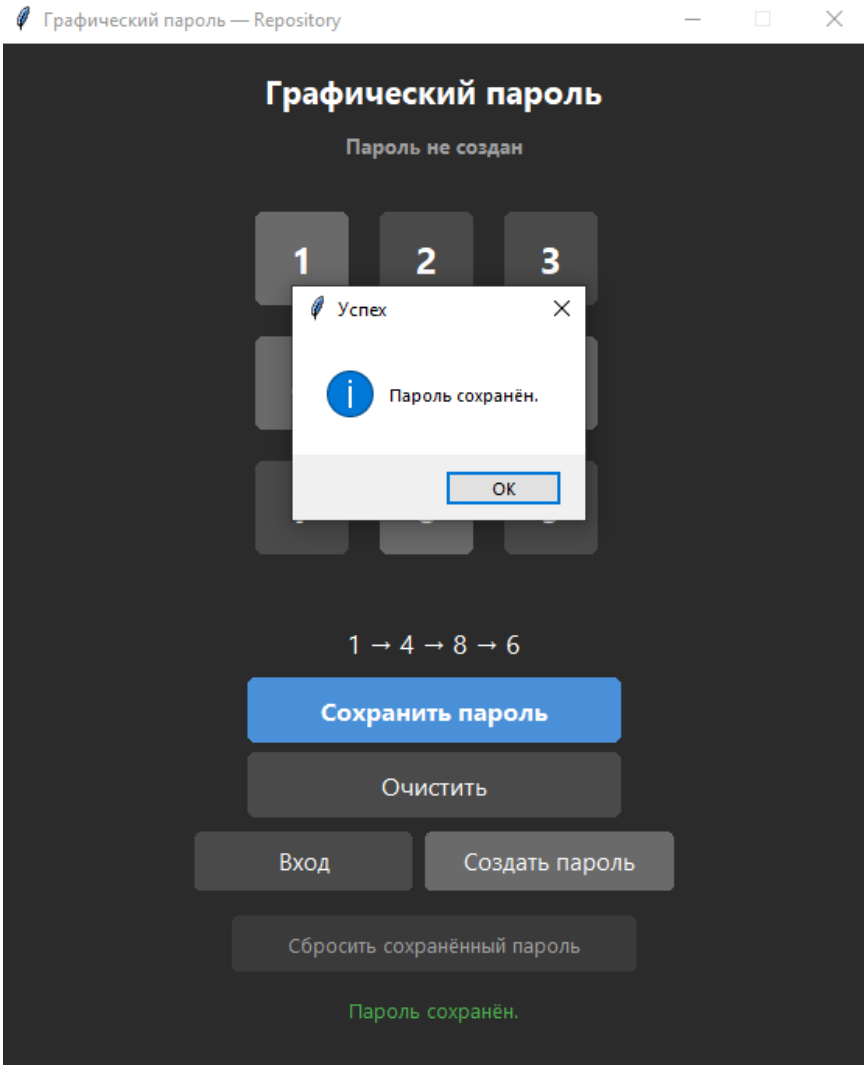


Рисунок 3 - Создание пароля.

Выбрана последовательность 1 → 4 → 8 → 6, нажата «Сохранить пароль». Диалог «Пароль сохранён». Метка показывает «Пароль создан. Неудачных попыток: 0».

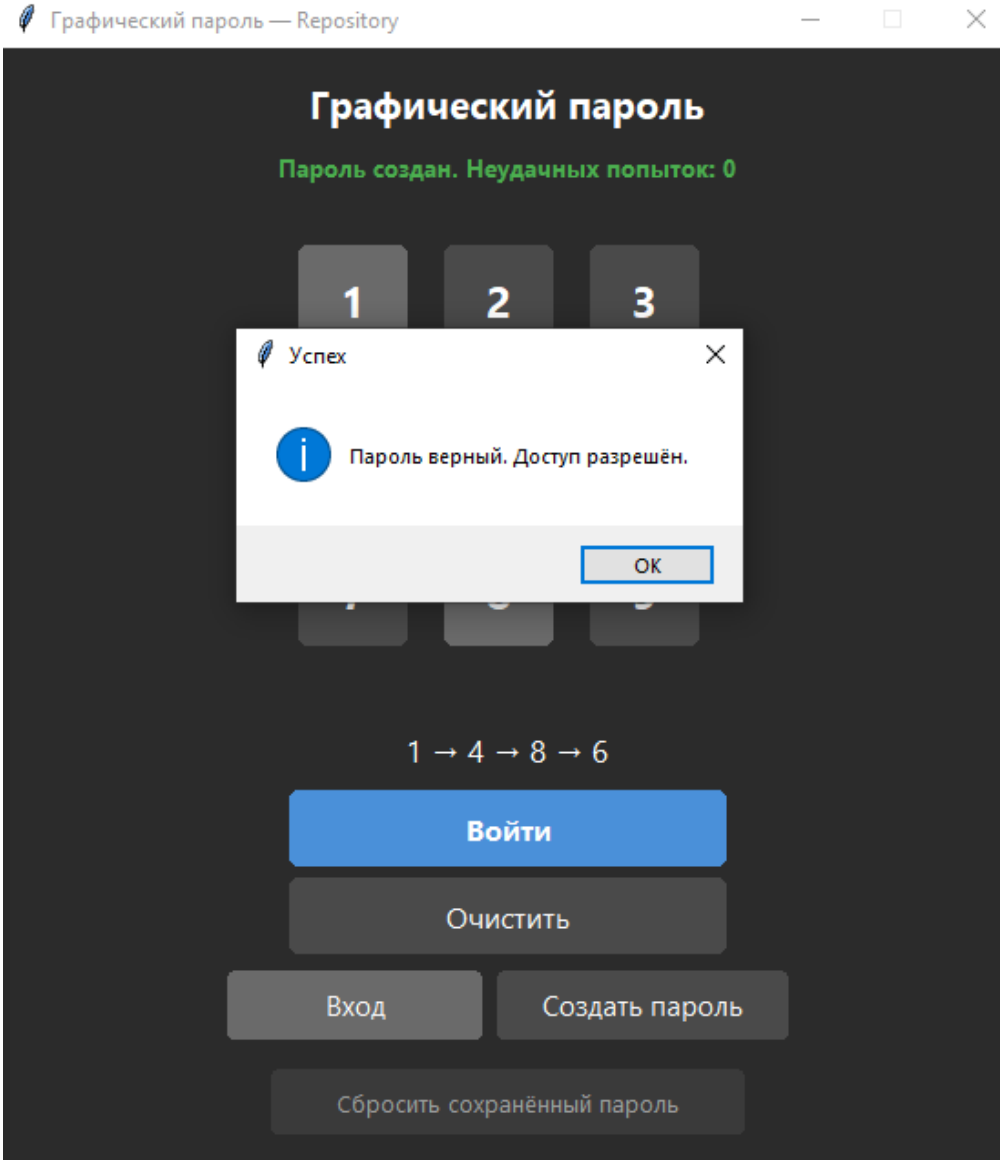


Рисунок 4 - Проверка правильного пароля.

Введена та же последовательность. Диалог «Пароль верный. Доступ разрешён.». Счётчик попыток остаётся 0.

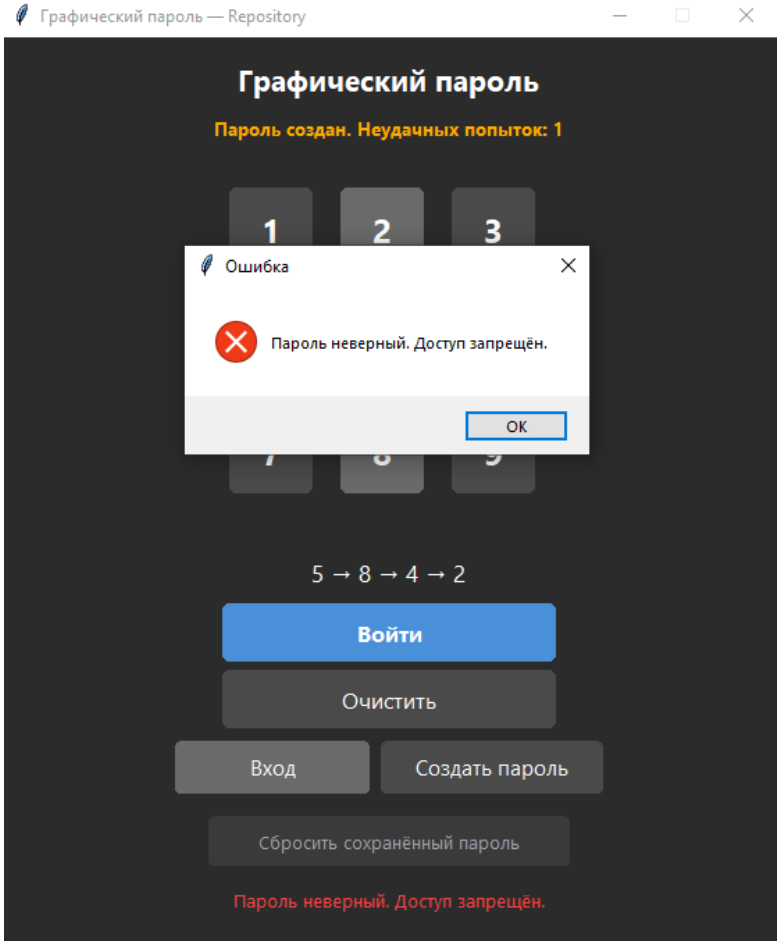


Рисунок 5 - Проверка неправильного пароля.

Введена 5 → 8 → 4 → 2. Диалог «Пароль неверный. Доступ запрещён.». Метка показывает «Неудачных попыток: 1».

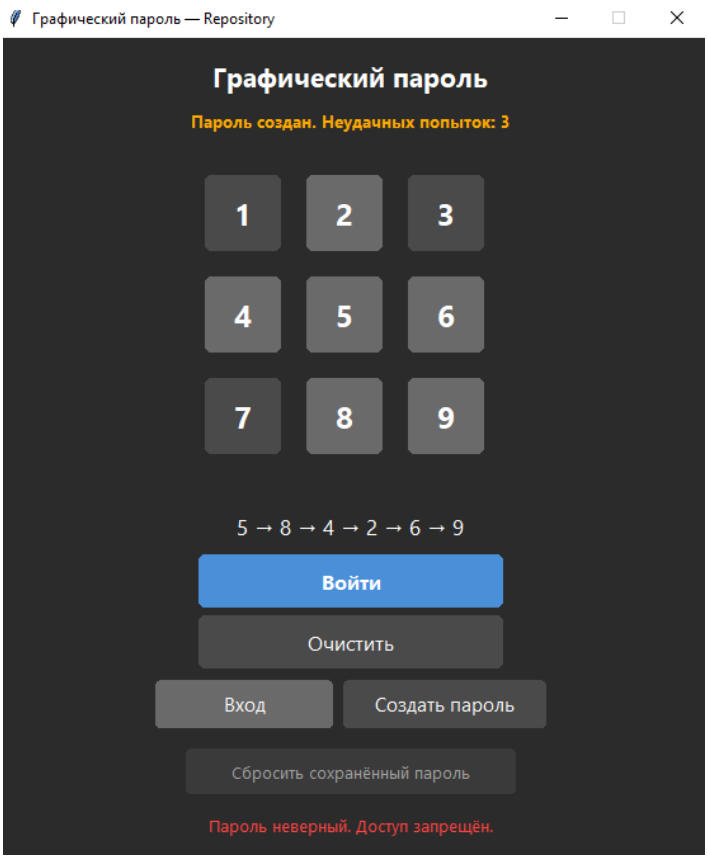


Рисунок 6 - Несколько неудачных попыток.

После трёх неверных вводов метка показывает «Неудачных попыток: 3».

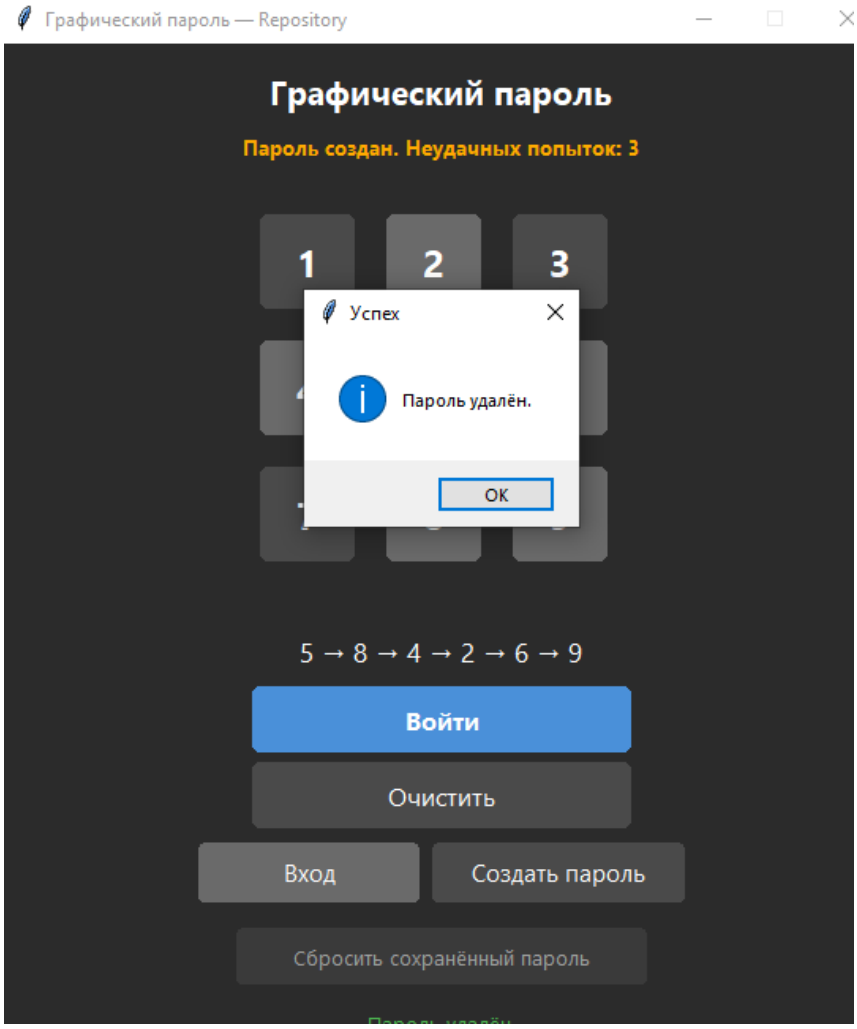


Рисунок 7 - Сброс пароля.

Нажата «Сбросить сохранённый пароль». Диалог «Пароль удалён.». Метка снова «Пароль не создан», файл graphical_password.json удалён.

Вывод

В ходе выполнения лабораторной работы был изучен и реализован системный паттерн Repository. Репозиторий PasswordRepository инкапсулирует работу с JSON-файлом и предоставляет клиенту интерфейс, похожий на коллекцию доменных объектов (PasswordRecord). PasswordService содержит только бизнес-логику и не зависит от способа хранения: замена репозитория на реализацию поверх SQLite или удалённого API не потребует изменений в сервисе и UI.

Паттерн хорошо сочетается с архитектурой предыдущих лабораторных работ (MVC в лабе 12, API Gateway + Circuit Breaker в лабе 13): контроллер - сервис - репозиторий образуют чистую слоистую структуру.